DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 4 — The RISC-V Instruction Set Architecture II: Control Flow, Procedures & the Stack

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

RECAP & THIS WEEK

From Straight Lines to Real Programs

Last week: we could compute and move data — but only in a straight line. This week we add the two things that make code a program: decisions, and procedures.

1	**Conditional Branches**	beq, bne, blt — and the B-type format
2	**Compiling if–else and Loops**	How high-level control becomes branches
3	**Jumps & the Link Register**	jal, jalr and the J-type format
4	**Procedures**	The six steps every call must perform
5	**The Stack**	Saving registers across a call
6	**Addressing Modes**	The complete picture, assembled

MAKING DECISIONS

Conditional Branches

A branch compares two registers and, if the test passes, changes where the processor goes next.

```
beq  s0, s1, Label   # branch if equal
bne  s0, s1, Label   # branch if not equal
blt  s0, s1, Label   # branch if less than
bge  s0, s1, Label   # branch if >=
```

There is no “compare” instruction that sets a flag, and no branch-if-greater. RISC-V simply swaps the operands instead — fewer instructions to build, decode and verify.

B-TYPE FORMAT



The immediate is a PC-relative offset, not an address — and it is scrambled across two fields so the sign bit stays at bit 31 for every format. Sign-extension can then begin before decoding finishes.

WORKED EXAMPLE

Compiling if – else

Note the inversion: the compiler branches on the opposite of the source condition, to skip over the then-block.

C SOURCE

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

RISC-V ASSEMBLY

```
     bne  s3, s4, Else   # skip if i != j
     add  s0, s1, s2     # f = g + h
     beq  x0, x0, Exit   # unconditional
Else:sub  s0, s1, s2     # f = g - h
Exit:
```

beq x0, x0, Exit is an unconditional jump in disguise — x0 always equals itself. One register, used cleverly, saves an entire instruction from the ISA.

WORKED EXAMPLE

Compiling a Loop

A loop is just a branch that points backwards.

C SOURCE

```
sum = 0;
for (i = 0; i < n; i++)
    sum += A[i];
```

Why slli t2, t1, 2?

Shifting left by 2 multiplies by 4 — converting an array index into a byte offset, far more cheaply than a multiply instruction.

RISC-V ASSEMBLY

```
      li   t0, 0          # sum
      li   t1, 0          # i
Loop: bge  t1, s2, Done   # exit if i >= n
      slli t2, t1, 2      # i * 4
      add  t2, s1, t2     # &A[i]
      lw   t3, 0(t2)
      add  t0, t0, t3     # sum += A[i]
      addi t1, t1, 1      # i++
      beq  x0, x0, Loop
Done:
```

GOING SOMEWHERE ELSE

Jumps and the Link Register

`jal rd, Label`

Jump and link

Jumps to Label, and stores the address of the following instruction in rd — normally ra. That saved address is how the procedure knows its way home.

`jalr rd, rs1, imm`

Jump and link register

Jumps to an address held in a register. This is how you return (jalr x0, ra, 0), and how switch statements and function pointers work.

J-TYPE FORMAT (jal)



A 20-bit PC-relative offset gives jal a reach of roughly ±1 MB — enough for almost any call inside a single program.

CALLING A PROCEDURE

Six Steps, Every Time

A procedure call is a contract between caller and callee. These six steps are that contract.

1. Place arguments where the procedure can reach them — a0–a7
2. Transfer control to the procedure — jal
3. Acquire the storage the procedure needs — the stack
4. Perform the task
5. Place the result where the caller can reach it — a0–a1
6. Return control to the point of origin — jalr via ra

THE REGISTER CONTRACT

Who Preserves What

With only 32 registers shared by every procedure, there must be a rule about who may destroy what. This is that rule.

PRESERVED ACROSS A CALL

```
s0–s11   saved registers
sp        stack pointer
ra        return address*
```

The callee must leave these exactly as it found them — saving and restoring them on the stack if it needs to use them.

NOT PRESERVED

```
t0–t6    temporaries
a0–a7    arguments / results
```

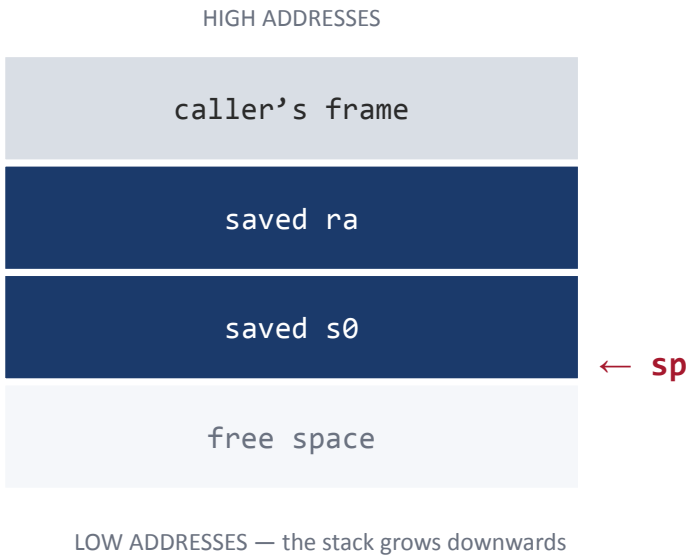
The callee may overwrite these freely. If the caller still needs a value here, the caller must save it before calling.

** ra is preserved by convention, but any procedure that itself makes a call must save it — the second jal would otherwise overwrite it.*

STORAGE THAT GROWS AND SHRINKS

The Stack

When registers run out, a procedure spills them to the stack — and restores them before returning.




```
leaf:
    addi sp, sp, -8      # make room
    sw   ra, 4(sp)       # save ra
    sw   s0, 0(sp)       # save s0

    # ... body of the procedure ...

    lw   s0, 0(sp)       # restore
    lw   ra, 4(sp)
    addi sp, sp, 8       # release
    jalr x0, ra, 0       # return
```

Push and pop are not instructions — they are a convention built from addi and sw/lw. Every allocation must be matched by a release, or the stack pointer drifts and the return fails.

THE COMPLETE PICTURE

RISC-V Addressing Modes

Having seen every instruction class, we can now name every way RISC-V forms an operand or a target address.

Mode	How the operand or address is formed	Example
Immediate	The operand is a constant inside the instruction	`addi t0, t1, 12`
Register	The operand is a register	`add t0, t1, t2`
Base + displacement	Address = register + constant offset	`lw t0, 8(s1)`
PC-relative	Target = PC + offset (branches and jal)	`beq s0, s1, Label`

Four modes — against dozens in some older architectures. Every one you leave out is hardware you never have to build, verify, or power.

THINK ABOUT IT

A procedure that calls itself must save its own return address — or it will never find its way back.

Recursion works only because each call gets a fresh stack frame. What happens when the recursion is too deep, or when a procedure writes past the space it allocated? Why is that failure both a correctness bug and a security vulnerability?

Discuss with a neighbour for two minutes before we continue.

HANDS-ON THIS WEEK

Lab 3 — Loops, Procedures & the Stack

Write a counted loop

Sum an array in memory, using slli to index it correctly.

Write a leaf procedure

Take arguments in a0–a1, return a result in a0, respect the conventions.

Save and restore

Extend it to call another procedure — and discover why ra must go on the stack.

DEBUGGING TIP

When a procedure returns to the wrong place, check sp first. Step through in RARS with the stack view open and watch sp before and after the call — it should hold exactly the same value.

Class Test 1 is in Week 6 and covers Weeks 1–5: performance analysis, the ISA, and assembly. This lab is your best preparation for it.

CHECKPOINT · CLO2

Key Takeaways

1. Translate if–else statements and loops into branches, and explain why the condition is inverted.
2. Use jal and jalr to call and return, and explain the role of the link register.
3. Write a procedure that respects the calling convention, saving and restoring registers on the stack.
4. Name the four RISC-V addressing modes and identify which instructions use each.

COMING UP — WEEK 5

Computer Arithmetic

Integer add, subtract, multiply, divide — and IEEE 754 floating point

